

Exercise 2 — Basics of React.js

5 small programs, written as simply as possible for first-time React learners.

How to Run Any Program Below

Step	Do this
1	Install Node.js from nodejs.org (LTS version)
2	Open terminal and run: <code>npm create vite@latest my-app -- --template react</code>
3	<code>cd my-app</code> then <code>npm install</code>
4	Open src/App.jsx , delete everything, paste the program
5	Run <code>npm run dev</code> and open the link shown (localhost:5173)

Note: If your lab uses Create React App instead, paste the code into **src/App.js** — the code is exactly the same.

3 Words You Must Know

Word	Simple meaning
Component	A reusable piece of the page (like a LEGO block)
JSX	HTML-like code written inside JavaScript
State	Data that can change. When it changes, the page updates itself

a) Counter Button using Class Component

AIM

To write a React program to implement a counter button using a class component.

PROGRAM (App.jsx)

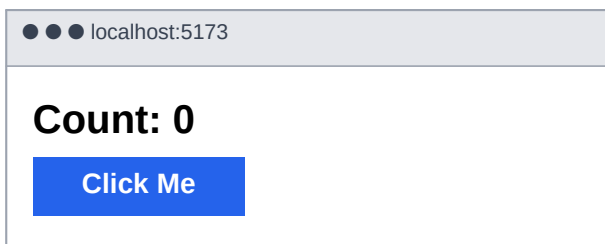
```
import React, { Component } from "react";

class App extends Component {
  state = { count: 0 };

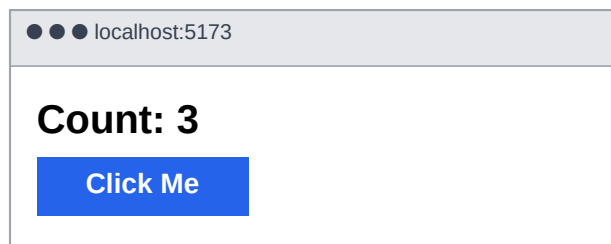
  render() {
    return (
      <div>
        <h1>Count: {this.state.count}</h1>
        <button onClick={() => this.setState({ count: this.state.count + 1 })}>
          Click Me
        </button>
      </div>
    );
  }
}

export default App;
```

OUTPUT



Before clicking



After clicking 3 times

KEY POINTS

- **class App extends Component** → makes a class component.
- **state = { count: 0 }** → counter starts at 0.
- **this.setState()** → changes the count, and the page updates.

b) Counter Button using Functional Component

AIM

To write a React program to implement a counter button using a functional component.

PROGRAM (App.jsx)

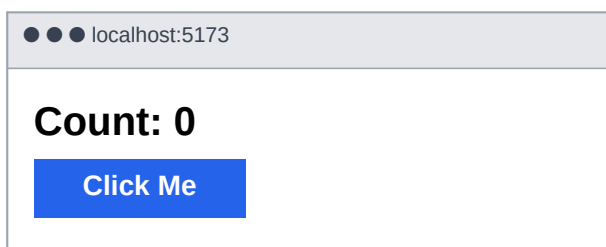
```
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

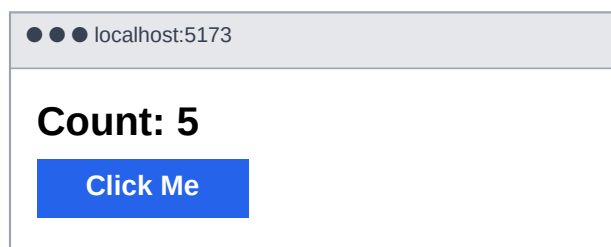
  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Click Me</button>
    </div>
  );
}

export default App;
```

OUTPUT



Before clicking



After clicking 5 times

KEY POINTS

- **useState(0)** → creates state with starting value 0.
- **count** = current value, **setCount** = function to change it.
- Same result as (a), but **shorter** — no class, no *this*.

Class vs Functional — quick comparison

	Class (a)	Functional (b)
Create state	state = { count: 0 }	useState(0)
Read value	this.state.count	count
Change value	this.setState(...)	setCount(...)
Code length	Longer	Shorter (modern way)

c) Handle Button Click Event in Functional Component

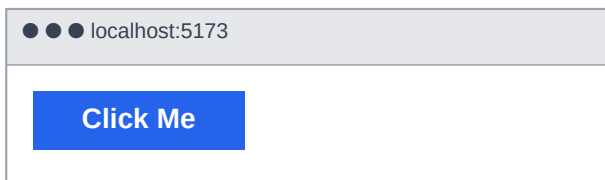
AIM

To write a React program to handle button click events in a functional component.

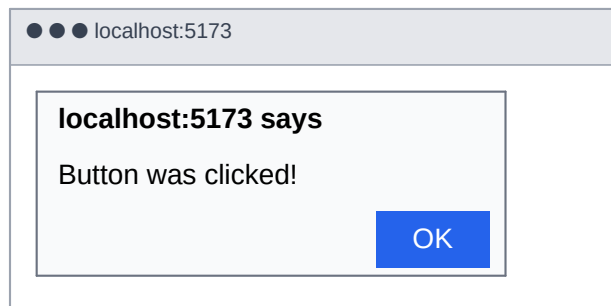
PROGRAM (App.jsx)

```
function App() {  
  function handleClick() {  
    alert("Button was clicked!");  
  }  
  
  return <button onClick={handleClick}>Click Me</button>;  
}  
  
export default App;
```

OUTPUT



Page loads



After clicking — popup appears

KEY POINTS

- **handleClick** → a normal function that runs on click.
- **onClick={handleClick}** → connects the button to the function.
- Write **handleClick**, NOT **handleClick()** — the () would run it immediately.

d) Conditional Rendering of a Component

AIM

To write a React program to conditionally render a component in the browser.

PROGRAM (App.jsx)

```
import { useState } from "react";

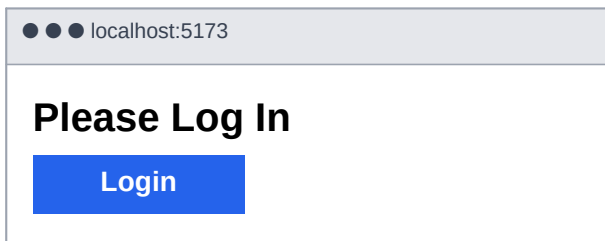
function Welcome() {
  return <h1>Welcome, User!</h1>;
}

function App() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

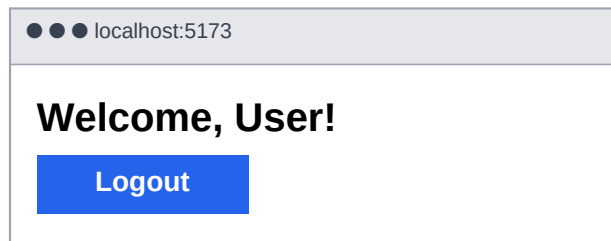
  return (
    <div>
      {isLoggedIn ? <Welcome /> : <h1>Please Log In</h1>}
      <button onClick={() => setIsLoggedIn(!isLoggedIn)}>
        {isLoggedIn ? "Logout" : "Login"}
      </button>
    </div>
  );
}

export default App;
```

OUTPUT



isLoggedIn = false



After clicking Login (true)

KEY POINTS

- **condition ? A : B** → if true show A, else show B.
- **Welcome** component shows **only** when isLoggedIn is true.
- **!isLoggedIn** → flips true to false and false to true.

e) Display Text using String Literals

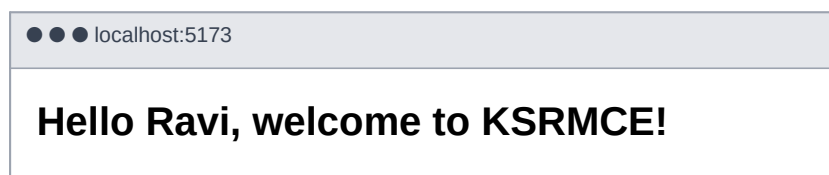
AIM

To write a React program to display text using string (template) literals.

PROGRAM (App.jsx)

```
function App() {  
  const name = "Ravi";  
  const college = "KSRMCE";  
  
  return <h1>{`Hello ${name}, welcome to ${college}!`}</h1>;  
}  
  
export default App;
```

OUTPUT



Output in browser

KEY POINTS

- String literals use **backticks** (``), not quotes.
- **`\${variable}`** → puts the variable's value inside the text.
- In JSX, wrap it in **{ }** so React treats it as JavaScript.

RESULT

All five React programs were written and executed successfully, and the outputs were verified in the browser.